

Optimizing Constrained Engineering Problems using Opposition based Swarm Algorithms

Rahul Katarya
Department of Computer Science and
Engineering
Delhi Technological University
New Delhi, India
rahuldtu@gmail.com

Utkarsh Agrawal
Department of Computer Science and
Engineering
Delhi Technological University
New Delhi, India
agrawal.utkarsh8@gmail.com

Vasudha Rohatgi
Department of Computer Science and
Engineering
Delhi Technological University
New Delhi, India
vasudha.rohatgi@gmail.com

Abstract— Real-world engineering problems involve complicated mathematical functions with strict constraints and are difficult to solve analytically due to their complexity. Conventional methods fail to handle problems with multiple local optimum values and thus cannot guarantee global optimum solutions. Two-Phase Opposition-based PSO-GWO (TPOPSOGWO) is a novel hybrid optimization method that combines the exploration and exploitation capabilities of the Grey Wolf Optimizer (GWO) and Particle Swarm Optimization (PSO) algorithms. The two optimizer techniques are carefully applied to the same search agents at different stages in this algorithm without changing the essential operations of the classical algorithms. When opposition-based learning is integrated to improve the diversity of the population, the system's performance increases even more. We evaluated the proposed approach's performance by comparing its mean and best function values to existing state-of-the-art methods. TPOPSOGWO surpasses all traditional swarm optimizers in many engineering issues, demonstrating its potential to aid industrial development whose challenges reside in limited and otherwise unexplored search areas.

Keywords— Engineering Problems, Grey Wolf Optimizer, Opposition-based learning, Particle Swarm Optimization

I. INTRODUCTION

Real-world engineering problems, for instance, structural optimization and engineering design, are difficult to solve analytically due to their complexity. In general, these problems involve complicated mathematical functions with strict constraints and many decision variables. There have been a variety of methods proposed and suggested over the last decade to solve these problems efficiently and effectively. Conventional methods, including numerical optimization methods, fail to handle problems with multiple local optimum values and thus cannot guarantee global optimum solutions.

Metaheuristic algorithms, which parametrically optimize existing heuristic search techniques [1-6], have been extensively developed and channeled over the past decade to solve complex optimization problems, which work by locating near-optimal solutions. Unlike conventional methods, these algorithms do not assume anything about the problem and do not involve calculating derivatives to operate. Swarm algorithms, a subset of metaheuristic algorithms, leverage the innate logic behind animal swarm behavior. Particle Swarm Optimization algorithm (PSO) [1], which is inspired by migratory birds, Grey Wolf Optimizer (GWO), which mimics the motivations and behavior

of a pack of grey wolves hunting for food [2], Whale search or Whale Optimization algorithm (WOA), which is inspired by the behavior of humpback whale herds [3], etc. are prime examples of metaheuristic swarm techniques.

Exploration and exploitation are the two main search behaviors employed by metaheuristic algorithms. The term exploration refers to the process of exhausting the whole search domain. Exploiting, on the other hand, requires locating the optimal solution in nearby search spaces. Any metaheuristic algorithm that seeks to improve the outcome must do so while sufficiently calculating the tradeoff between exploration and exploitation capabilities. This is what sets current algorithms apart from one another. Any swarm algorithm with weak exploration and exploitation capabilities can quickly become caught in local minima. An exploration-exploitation imbalance plagues existing swarm techniques. Although PSO has a remarkable exploitation ability, it struggles to explore in high-dimensional space and often gets stuck in local minima. In contrast, while GWO has excellent exploration capabilities, it suffers from poor exploitation compared to the same capacity of other state-of-the-art swarm optimizers.

We present a novel and straightforward method for optimizing well-known and complex constrained engineering problems using an intuitive combination of the PSO and GWO algorithm, named as Two-Phase Opposition-based PSO-GWO (TPOPSOGWO). The algorithm combines the exploitation and exploration abilities of PSO and GWO, respectively. Incorporating Opposition-based learning for population initialization supplements our algorithm to produce a better convergence rate and search space development. Contrary to previous algorithms using swarms, our methodology demonstrates simplicity, replicability, and the highest performance we have ever seen. As a summary, our paper makes the following contributions:

- We proposed a novel method that combines the algorithmic capabilities of the PSO and GWO.
- We employed opposition-based learning to improve the diversity of the population instead of random initialization.
- We assessed the performance of the proposed approach by comparing its mean values and best function values with those corresponding to existing state-of-the-art

swarm optimizers for complex constrained engineering problems.

- The proposed method outperforms all conventional swarm optimizers, showing potential to assist industrial development whose challenges lie in constrained and otherwise unknown search territories.

The following study is organized into seven discrete sections; Section II details the literature review on swarm intelligence to solve optimization problems, Section III details the specific swarm methods and initialization methods we have leveraged, Section IV details the final formulation and algorithm of our proposed technique, Section V introduces some challenging constrained engineering problems, Section VI illustrates experimental results and related comparisons and Section VII summarizes the experiment.

II. RELATED WORK

Researchers have developed various methods to tackle some of the most challenging engineering optimization problems that are often encountered in the design of real-world systems. Toklu et al. [7] proposed Total Potential Optimization using Metaheuristic Algorithms. This technique has shown to be reliable in addressing nonlinear engineering design problems such as n-bar trusses, real-time tensegrity systems, plane stress systems, and complex cable networks. In their research on optimizing constrained engineering design problems, Oliva et al. [8] devised OBMSA, a breakthrough variant of the well-known moth swarm algorithm that initializes the search agents through opposition-based learning (OBL). Innovating in reservoir operation optimization, Dahmani et al. [9] used the HPSOGWO algorithm previously proposed by Zhou et al.. They modified it to generate superior quality and better-adapted solutions compared to preexisting tools like Real-Coded Genetic Algorithms and Gravitational Search Algorithms, often employed for the same task. Sattar et al. [10] presented the Smart Flower Optimization Algorithm, a novel form of a botanically-influenced optimization algorithm that leverages nature's development principles to improve and optimize complicated structural conundrums. The SFOA is assessed using four complex and multi-constraint civil design problems, some of which we have also tested against our proposed algorithm. Premkumar et al. [11] devised a crossbred model trained to determine the unknown parameters from a P.V. single-diode model. Two parameters were modeled using standard test conditions, and the remaining three parameters were streamlined using the PSOGWO algorithm. Developed by Zhang et al. [12], a new strategy called differential perturbation was devised and adapted into a simplified GWO, resulting in improved global search capability of the GWO technique without compromising exploitation capability. The inherent shortcoming of swarm algorithms, local minima, is remedied by incorporating a stochastic mean example learning method into the algorithm to generate Mean Example Learning PSO (MELPSO). Finally, a hybrid algorithm that can employ both SDPGWO and MELPSO is proposed to create a worldwide and extendable search strategy. It uses both the algorithms without mutual conflict - a poor-for-change method that seeks to harmonize SDPGWO and MELPSO to operate in tandem to generate optimal search results.

A new streamlined convergent hybrid algorithm has been proposed by Singh et al. [13] that combines PSO and GWO, so they work simultaneously but utilize each other in particular phases. Zhang et al. [14] ideated a combinatorial algorithm of Cuckoo Search with Differential Evolution (CSDE) to solve restrictive structural design challenges, which showed great potential even with multimodal systems. Although the proposed algorithm has advantages like fewer parameters and good global search, it suffers from premature convergence. In their study, Dhiman et al. [15] recommended a merger technique based on the migratory-foraging behavior of Emperor Penguin tribes and Salp chain-behavior inspired Swarm Algorithms, which is evaluated using six constrained structural-engineering problems. Sayed et al. [16] created a hybrid method that combines Simulated Annealing and Moth Flame Optimization, termed SA-MFO. The algorithm has fast searching and the ability to escape from local minima, and its performance is evaluated on four constrained engineering problems. To expand the search space and improve convergence rates, Debnath et al. [17] proposed a centralized memory-based dragonfly algorithm that leverages the strength of differential evolution. An assessment of the algorithm's performance is also made using classical, constrained structural design problems.

Tizhoosh [18] proved that opposition-based learning works by explaining that an opposite number is closer to the optimal value than a random number, thus improving the searchability and convergence rate. Kang et al. [19] utilized the technique by incorporating opposition-based learning in the conventional PSO algorithm. Employing elite opposition-based learning, Zhang et al. [20] improved the efficacy of GWO using classical metaheuristics.

III. OPTIMIZATION AND INITIALIZATION METHODS

A. PSO

PSO is one of the oldest and most potent metaheuristic methods for locating optimal solutions in high-dimensional spaces [1]. The search agents' initial positions are induced at random, and their positions are renewed using the formula described in [1].

$$\vec{x}_{n+1}^i = \vec{x}_n^i + \vec{v}_{n+1}^i \quad (1)$$

$$\vec{v}_{n+1}^i = \omega \vec{v}_n^i + c_1 u_1 (\vec{p}_n^i - \vec{x}_n^i) + c_2 u_2 (\vec{p}_n^g - \vec{x}_n^i) \quad (2)$$

Here i refers to the search agent, n is the iteration number, u_1 and u_2 are random numbers in the range [0,1], ω is the inertia weight parameter, and c_1, c_2 are optimization parameters. $\vec{x}$ represents the location of the particle agent, $\vec{v}$ characterizes its velocity vector, $\vec{p}^i$ denotes the personal best position of the i 'th search agent, $\vec{p}^g$ is the most optimal or global best position of all search agents. All particles' individual and global best positions are recorded in the memory to be revised after each cycle.

B. GWO

GWO algorithm draws inspiration from grey wolves' hunting behavior [2].

The Grey wolf regime is represented by alpha, beta, delta, and omega wolf classes. In decreasing quality order, the alpha,

beta, and delta wolves, which are prime hunters, symbolize the best possible solutions. Following the alpha, beta, and delta wolves, omega wolves are representative of other acceptable outcomes. The encircling behavior of the wolves is represented mathematically as:

$$\vec{P}(n+1) = \vec{P}_p(n) - \vec{A} \cdot |\vec{C} \cdot \vec{P}_p(n) - \vec{P}(n)| \quad (3)$$

where n denotes the iteration number, $\vec{P}_p$ represents the position of the prey and $\vec{P}$ denotes the position of the grey wolves. The coefficients A and C are evaluated as given below:

$$\vec{A} = \vec{a} \cdot (2\vec{k}_1 - I) \quad (4)$$

$$\vec{C} = 2\vec{k}_2 \quad (5)$$

The vector $\vec{a}$ is decremented from 2 to 0 over the total number of iterations linearly. This is done to regulate and guide the exploratory and exploitative behavior of the wolves. $\vec{k}_1$ and $\vec{k}_2$ are arbitrary values generated from the interval $[0,1]$.

The positions of the wolves employed can be updated using the following equations:

$$\vec{P}_1 = \vec{P}_\alpha - \vec{A}_1 \cdot |\vec{C}_1 \cdot \vec{P}_\alpha - \vec{P}| \quad (6)$$

$$\vec{P}_2 = \vec{P}_\beta - \vec{A}_2 \cdot |\vec{C}_2 \cdot \vec{P}_\beta - \vec{P}| \quad (7)$$

$$\vec{P}_3 = \vec{P}_\delta - \vec{A}_3 \cdot |\vec{C}_3 \cdot \vec{P}_\delta - \vec{P}| \quad (8)$$

$$\vec{P}(n+1) = \frac{\vec{P}_1 + \vec{P}_2 + \vec{P}_3}{3} \quad (9)$$

where $\vec{P}(n+1)$ is the new position of the prey, and $\vec{P}_\alpha, \vec{P}_\beta$ and $\vec{P}_\delta$ are the positions of alpha, beta, and delta wolves, respectively.

C. Opposition-based Learning

Metaheuristic algorithms typically initialize the position vectors of the population members with arbitrary numbers generated from the minimum and maximum bounds of the search space. In most cases, these algorithms are slow to converge. We generate both random solutions and their opposite solutions simultaneously to counter this issue. This improves the chances of arriving at the optimal solution.

Let $P = \{x_1, x_2, \dots, x_d\}$ be the positions of a particle in a D dimensional search arena, where x_i ($1 \leq i \leq D$) is a value within the bounds of a_i and b_i . The reverse or opposed position vector $\check{P} = \{\check{x}_1, \check{x}_2, \dots, \check{x}_D\}$ can be determined from the following equation:

$$\check{x}_i = a_i + b_i - x_i \quad (10)$$

IV. PROPOSED METHOD

The Two-Phase Opposition-based PSO-GWO algorithm has been configured to be an amalgamation of PSO and GWO that preserves the specific logic of both algorithms.

In conventional swarm algorithms, initial population $X = \{x_{p1}, x_{p2}, \dots, x_{pq}, \dots, x_{pS}\}$ ($p = 1, 2, \dots, NP$; $q = 1, 2, \dots, S$) is generated randomly. We employ Opposition-based Learning to

generate both random population X and opposite population $\check{X}$ simultaneously from (10). The fitness of both the populations are calculated, which allows us to select only the best $N.P.$ initial positions from $\{X_i, \check{X}_i\}$, where $N.P$ is the population size, which acts as the initial population.

A new parameter named phase factor (p) is introduced in the proposed TPOPSOGWO algorithm. The parameter balances and controls the number of iterations required by the GWO phase to be executed for better exploration.

After initializing the population, the first phase of the proposed TPOPSOGWO directs the search agents and updates their position using the GWO algorithm. Search agents navigate the search space more effectively owing to the function of GWO. This is followed by directing the same search agents and updating their positions using the PSO algorithm. Because of PSO's extensive search capability, the proposed technique can converge to the best possible result in the restricted search region.

Our method can be described algorithmically, as shown in Fig.1.

V. CONSTRAINED ENGINEERING PROBLEMS

Some commonly employed industrial and civil engineering elements, namely Cantilever Beam Design (P1) [21], Tension/Compression Spring Design (P2) [22], Three-bar-truss Design (P3) [23], Pressure Vessel Design (P4) [24], and Welded Beam Design (P5) [25] with the load constraints faced in their manufacture and utility are discussed in this section.

A. Cantilever Beam Design Problem

Cantilever beams pose a load distribution challenge, mandating the minimization of the total volume and weights of components of the beam. Considering a five-stepped beam, we target ten design variables whose optimal values need to be calculated to minimize the total volume, employing the function:

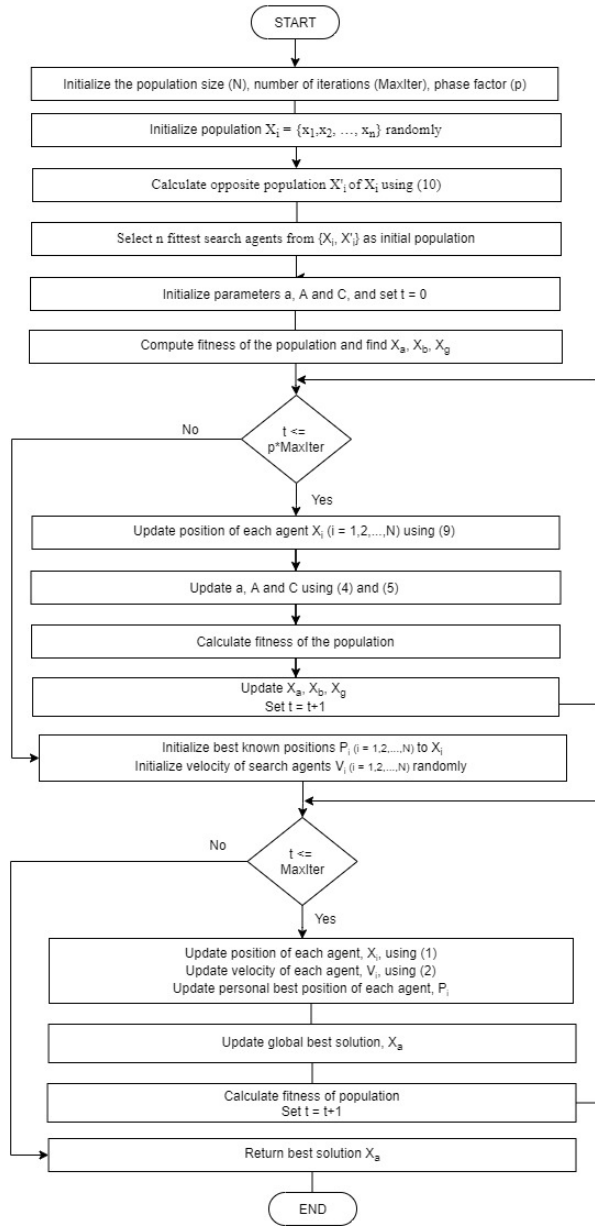


Fig 1. Proposed TPOPSOGWO algorithm

$$\text{Minimize: } f(z) = 6.24 \times 10^{-2} (z_1 + z_2 + z_3 + z_4 + z_5) \quad (11)$$

$$\text{S.t.: } c_1 = \left(\frac{61}{z_1} + \frac{37}{z_2} + \frac{19}{z_3} + \frac{7}{z_4} + \frac{1}{z_5} \right) - 1 \leq 0$$

$$10^{-2} \leq z_i \leq 10^2 \quad i=1,2,3,4,5$$

B. Spring Design Problem

This structural element design requires minimizing the weight of the spring, given constraints on diameter, shear, and surge frequency. The problem is defined as:

$$\text{Minimize: } f(z) = (z_3 + 2)z_2z_1^2 \quad (12)$$

S.t.:

$$c_1 = 1 - \frac{z_2^3 z_3}{7.1785 \times 10^4 z_1^4 z_1^4} \leq 0$$

$$c_2 = 4z_2^2 - \frac{z_1 z_2}{1.2566 \times 10^4 (z_2 z_1^3 - z_1^4)} + \frac{1}{5.108 \times 10^3 z_1^2} - 1 \leq 0$$

$$c_3 = 1 - \frac{1.4045 \times 10^2 z_1}{z_2^2 z_3} \leq 0$$

$$c_4 = z_2 + \frac{z_1}{1.5} - 1 \leq 0$$

$$5 \times 10^{-2} \leq z_1 \leq 2, \quad 2.5 \times 10^{-1} \leq z_2 \leq 1.3, \quad 2 \leq z_3 \leq 1.5 \times 10^1$$

C. Three-bar-truss Design Problem

A truss is a structural configuration in which bars are connected at joints load through the axial, and the configuration carries force in the bars. This problem involves minimizing the weight of the material of the members involved. The problem can be formulated using the following equations:

$$\text{Minimize: } f(z) = (2\sqrt{2}z_1 + z_2) \times 10^2 \quad (13)$$

S.t.:

$$c_1 = \frac{\sqrt{2}z_1 + z_2}{\sqrt{2}z_1^2 + 2z_1 z_2} \times 2 - 2 \leq 0$$

$$c_2 = \frac{x_2}{\sqrt{2}z_1^2 + 2z_1 z_2} \times 2 - 2 \leq 0$$

$$c_3 = \frac{1}{\sqrt{2}z_2 + z_1} \times 2 - 2 \leq 0$$

$$0 \leq z_1, z_2 \leq 1$$

D. Pressure Vessel Design Problem

Pressure vessels must be designed to hold large masses of gas and fluid, often toxic or flammable in nature. The main objective of this design involves economizing the total cost of building such a container while prioritizing its utility to store and transport high-pressure fluids safely. The problem can be represented with the equation below:

Minimize:

$$f(z) = 6.224 \times 10^{-1} z_1 z_3 z_4 + 1.7781 z_2 z_3^2 + 3.1661 z_1^2 z_4 + 19.84 z_1^2 z_3 \quad (14)$$

S.t.:

$$c_1 = -z_1 + 1.93 \times 10^{-2} z_3 \leq 0$$

$$c_2 = -z_2 + 9.54 \times 10^{-3} z_3 \leq 0$$

$$c_3 = -\pi z_3^2 z_4 - \frac{4}{3\pi z_3^3} + 1.296 \times 10^6 \leq 0$$

$$c_4 = z_4 - 2.4 \times 10^2 \leq 0$$

$$0 \leq z_i \leq 10^2 \quad i=1,2, \quad 10 \leq z_i \leq 2 \times 10^2 \quad i=3,4$$

E. Welded Beam Design Problem

This structural design problem involves reducing the total cost of constructing a welded beam while calculating the member dimensions to carry a specific load (P) within some logistically determining upper and lower bounds. The problem can be represented using the equation mentioned below:

Minimize:

$$f(z) = 1.1047z_1^2z_2 + 4.811 \times 10^{-2}z_3z_4(1.4 \times 10^1 + z_2) \quad (15) \quad 10^{-1} \leq z_1, z_4 \leq 2 \times 10^1, 10^{-1} \leq z_2, z_3 \leq 1 \times 10^1$$

S.t.:

$$c_1 = s(z) - s_{\max} \leq 0$$

$$c_2 = bs(z) - bs_{\max} \leq 0$$

$$c_3 = z_1 - z_4 \leq 0$$

$$c_4 = 1.0471 \times 10^{-1}z_1^2z_2 + 4.811 \times 10^{-2}z_3z_4(14 + z_2) - 5 \leq 0$$

$$c_5 = 1.25 \times 10^{-1} - z_1 \leq 0$$

$$c_6 = \sigma(z) - \sigma_{\max} \leq 0$$

$$c_7 = P-PC(z) \leq 0$$

VI. EXPERIMENTAL RESULTS AND DISCUSSION

We chose five well-known and complex engineering problems to evaluate the performance of the proposed algorithm, which were previously addressed in Section V. The results are validated and compared with other classical swarm optimizers, including PSO, GWO, WOA, Artificial Bee Colony (ABC) [4] algorithm, Dragonfly Algorithm (DA) [5], and Salp Swarm Algorithm (SSA) [6]. The algorithm code was created using MATLAB R2019a, installed on a Microsoft Windows 10 (64-bit) computer with an Intel i7 CPU and 16 G.B. of RAM. For each iteration, the population size in the solution space is set to 30, and the algorithm termination criteria are set to 500

TABLE I
COMPARISON OF RESULTS OF TPOPSOGWO WITH OTHER METHODS ON 6 CLASSICAL ENGINEERING PROBLEMS

Problem	Metrics	TPOPSOGWO	WOA	DA	GWO	ABC	PSO	SSA
P1	Avg	1.339989E+00	1.606740E+00	1.386444E+00	1.340136E+00	1.372641E+00	1.340045E+00	1.340072E+00
	Std	2.476986E-05	2.401720E-01	1.365018E-01	1.108036E-04	1.937872E-02	6.435995E-05	1.387358E-04
	Best	1.339957E+00	1.354738E+00	1.340817E+00	1.339977E+00	1.340810E+00	1.339961E+00	1.339961E+00
	Worst	1.340079E+00	2.947830E+00	2.371472E+00	1.340527E+00	1.428464E+00	1.340249E+00	1.340826E+00
P2	Avg	1.289525E-02	1.365273E-02	1.667505E-02	1.279711E-02	1.285094E-02	1.312534E-02	1.522132E-02
	Std	3.288460E-04	1.205970E-03	1.121246E-02	1.521020E-04	1.597793E-04	4.890382E-04	2.535794E-03
	Best	1.266523E-02	1.266530E-02	1.267246E-02	1.267412E-02	1.268615E-02	1.266527E-02	1.266545E-02
	Worst	1.447757E-02	1.779496E-02	9.699033E-02	1.352468E-02	1.342023E-02	1.473879E-02	2.495715E-02
P3	Avg	2.638965E+02	2.663968E+02	2.639197E+02	2.640925E+02	2.639339E+02	2.638968E+02	2.639017E+02
	Std	8.978020E-04	4.360250E+00	4.149569E-02	1.893968E+00	3.055655E-02	1.268873E-03	1.391068E-02
	Best	2.638958E+02	2.638959E+02	2.638958E+02	2.638960E+02	2.638974E+02	2.638958E+02	2.638958E+02
	Worst	2.638997E+02	2.828427E+02	2.641654E+02	2.828427E+02	2.640562E+02	2.639039E+02	2.639895E+02
P4	Avg	6.206890E+03	1.294746E+04	9.880727E+03	6.130565E+03	6.046008E+03	6.289329E+03	1.168275E+04
	Std	3.356240E+02	1.012038E+04	2.589383E+04	4.173280E+02	9.232938E+01	2.949326E+02	8.775780E+03
	Best	5.886840E+03	6.358038E+03	5.947293E+03	5.891689E+03	5.910802E+03	5.900828E+03	5.940807E+03
	Worst	7.238445E+03	6.379228E+04	2.651837E+05	7.339375E+03	6.355772E+03	7.284452E+03	6.374350E+04
P5	Avg	1.725392E+00	2.780472E+00	1.845704E+00	1.729509E+00	1.992981E+00	1.727542E+00	2.644077E+00
	Std	2.037374E-03	9.754811E-01	1.348013E-01	2.422320E-03	1.269135E-01	7.719851E-03	3.916617E-01
	Best	1.724853E+00	1.817677E+00	1.726613E+00	1.726080E+00	1.762221E+00	1.724853E+00	1.763883E+00
	Worst	1.738662E+00	5.817353E+00	2.497845E+00	1.740027E+00	2.299314E+00	1.772801E+00	3.739983E+00

TABLE II
ASSESSMENT OF RESULTS OF TPOPSOGWO WITH OTHER ALGORITHMS ORDERED BY RANKS

	Metrics	TPOPSOGWO	WOA	DA	GWO	ABC	PSO	SSA
P1	Avg	1	7	6	4	5	2	3
	Best	1	7	6	4	5	2	2
P2	Avg	3	5	7	1	2	4	6
	Best	1	3	5	6	7	2	4
P3	Avg	1	7	4	6	5	2	3
	Best	1	5	1	6	7	1	1
P4	Avg	3	7	5	2	1	4	6
	Best	1	7	6	2	4	3	5
P5	Avg	1	7	4	3	5	2	6
	Best	1	7	4	3	5	1	6
Overall	Avg	1.8	6.6	5.2	3.2	3.6	2.8	4.8
	Best	1	5.8	4.4	4.2	5.6	1.8	3.6

iterations. TPOPSOGWO is evaluated and executed on many possible values of the parameter phase factor to determine the optimum number of iterations required for executing the GWO phase in TPOPSOGWO. The best results were obtained for a phase factor value of 0.2. Parameters for all swarm optimizers are selected according to the values proposed in earlier literature. We set the required iterations of the model for each algorithm as

10 with the interest of maintaining bias-free results and hence tabulate the best and worst scores achieved by each algorithm after it runs entirely. In Table I, the results of the experiments performed with P1, P2, P3, P4, P5, and P6 are tabulated. Following observations are made from the results obtained:

- TPOPSOGWO is observed to outperform all the swarm optimizers in terms of achieving the best results for all the engineering problems.
- TPOPSOGWO has the best average score amongst all the algorithms for 3 out of 5 engineering problems.
- TPOPSOGWO fails to outperform all the methods for two engineering problems. It underperforms in terms of the average value, with GWO and ABC obtaining better results for P2 and P5, respectively. However, TPOPSOGWO manages to outperform all the methods in terms of best scores for the same engineering problems.

The performance of TPOPSOGWO is further compared with the same swarm optimizers in terms of the overall rank achieved. The results are tabulated in Table II. Following observations are made from the results obtained:

- TPOPSOGWO leads the rank with an overall rank of perfect 1 in obtaining the best value, followed by PSO and SSA with ranks of 1.8 and 3.6, respectively.
- TPOPSOGWO leads the rank with an overall rank of 1.8 in obtaining average value, followed by PSO and GWO with ranks of 2.8 and 3.2, respectively.
- TPOPSOGWO has overall outperformed both PSO and GWO, clearly indicating the success of our hybrid approach, enhancing both the exploration and exploitation ability of PSO and GWO, respectively.

Based on the obtained results of TPOPSOGWO as represented in Table I and II, it can be clearly stated that the changes incorporated has made TPOPSOGWO more relevant for constrained engineering problems and is a highly competitive method with great potential.

VII. CONCLUSION AND FUTURE WORK

This study proposes a novel composite model that combines the algorithmic capabilities of the PSO and GWO. We expect to avoid early convergence by embedding the exploratory and exploitative competencies of the PSO and GWO algorithms, respectively. Incorporating an opposition-based learning approach for smarter initialization of the search-populace in the solution space further improves the performance. GWO methods are employed in the first step of the algorithm to more thoroughly navigate the search space. From that, PSO recalculates the positions of the search agents in an iterative process that occurs until the termination period is attained. On a variety of multifaceted industrial issues, the efficacy of our strategy is compared to that of different modern, sophisticated swarm optimizers. According to the results of the experiments, TPOPSOGWO performed distinguishably better than other prominent algorithms when it was applied to the optimization of complex engineering problems.

TPOPSOGWO does not modify the mathematical equations which govern the logic used in updating the positions of the population members. This algorithm, therefore, has considerable potential for improvement and scalability. Furthermore, this simple search technique can be tailored to

elucidate other real-world problems and is a suitable candidate algorithm to be applied to multi-objective problems. The exploratory behavior of the proposed algorithm can be further improved by integrating crossover and mutation operations. Since TPOPSOGWO is a hybrid algorithm, the convergence of the algorithm has a scope of further improvement for better results. The influence of the new phase parameter can be studied and explored further in the future. The approach followed for integrating the PSO and GWO algorithms can be applied to other algorithms in further studies.

REFERENCES

- [1] J. Kennedy and R. Eberhart, "Particle Swarm Optimization," *IEEE International Conference on Neural Networks*, pp. 1942–1948, 1995.
- [2] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey Wolf Optimizer," *Advances in Engineering Software*, vol. 69, pp. 46–61, 2014, doi: 10.1016/j.advengsoft.2013.12.007.
- [3] S. Mirjalili and A. Lewis, "The Whale Optimization Algorithm," *Advances in Engineering Software*, vol. 95, pp. 51–67, May 2016, doi: 10.1016/j.advengsoft.2016.01.008.
- [4] D. Karaboga and B. Basturk, "A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm," *Journal of Global Optimization*, vol. 39, no. 3, pp. 459–471, Nov. 2007, doi: 10.1007/s10898-007-9149-x.
- [5] S. Mirjalili, "Dragonfly algorithm: a new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems," *Neural Computing and Applications*, vol. 27, no. 4, pp. 1053–1073, May 2016, doi: 10.1007/s00521-015-1920-1.
- [6] S. Mirjalili, A. H. Gandomi, S. Z. Mirjalili, S. Saremi, H. Faris, and S. M. Mirjalili, "Salp Swarm Algorithm: A bio-inspired optimizer for engineering design problems," *Advances in Engineering Software*, vol. 114, pp. 163–191, Dec. 2017, doi: 10.1016/j.advengsoft.2017.07.002.
- [7] Y. C. Toklu *et al.*, "Total potential optimization using metaheuristic algorithms for solving nonlinear plane strain systems," *Applied Sciences (Switzerland)*, vol. 11, no. 7, Apr. 2021, doi: 10.3390/app11073220.
- [8] D. Oliva *et al.*, "Opposition-based moth swarm algorithm," *Expert Systems with Applications*, vol. 184, Dec. 2021, doi: 10.1016/j.eswa.2021.115481.
- [9] S. Dahmani and D. Yebdri, "Hybrid Algorithm of Particle Swarm Optimization and Grey Wolf Optimizer for Reservoir Operation Management," *Water Resources Management*, vol. 34, no. 15, Springer Science and Business Media B.V., pp. 4545–4560, Dec. 01, 2020, doi: 10.1007/s11269-020-02656-8.
- [10] D. Sattar and R. Salim, "A smart metaheuristic algorithm for solving engineering problems," *Engineering with Computers*, vol. 37, no. 3, pp. 2389–2417, Jul. 2021, doi: 10.1007/s00366-020-00951-x.
- [11] M. Premkumar, R. Sowmya, S. Umashankar, and P. Jangir, "Extraction of uncertain parameters of single-diode photovoltaic module using hybrid particle swarm optimization and grey wolf optimization algorithm," in *Materials Today: Proceedings*, 2020, vol. 46, pp. 5315–5321, doi: 10.1016/j.matpr.2020.08.784.
- [12] X. Zhang, Q. Lin, W. Mao, S. Liu, Z. Dou, and G. Liu, "Hybrid Particle Swarm and Grey Wolf Optimizer and its application to clustering optimization," *Applied Soft Computing*, vol. 101, Mar. 2021, doi: 10.1016/j.asoc.2020.107061.
- [13] N. Singh and S. B. Singh, "Hybrid Algorithm of Particle Swarm Optimization and Grey Wolf Optimizer for Improving Convergence Performance," *Journal of Applied Mathematics*, vol. 2017, 2017, doi: 10.1155/2017/2030489.
- [14] Z. Zhang, S. Ding, and W. Jia, "A hybrid optimization algorithm based on cuckoo search and differential evolution for solving constrained engineering problems," *Engineering Applications of Artificial Intelligence*, vol. 85, pp. 254–268, Oct. 2019, doi: 10.1016/j.engappai.2019.06.017.
- [15] G. Dhiman, "ESA: a hybrid bio-inspired metaheuristic optimization approach for engineering problems," *Engineering with Computers*, vol. 37, no. 1, pp. 323–353, Jan. 2021, doi: 10.1007/s00366-019-00826-w.
- [16] G. I. Sayed and A. E. Hassanien, "A hybrid SA-MFO algorithm for function optimization and engineering design problems," *Complex &*

- Intelligent Systems*, vol. 4, no. 3, pp. 195–212, Oct. 2018, doi: 10.1007/s40747-018-0066-z.
- [17] S. Debnath, S. Baishya, D. Sen, and W. Arif, "A hybrid memory-based dragonfly algorithm with differential evolution for engineering application," *Engineering with Computers*, vol. 37, no. 4, pp. 2775–2802, Oct. 2021, doi: 10.1007/s00366-020-00958-4.
 - [18] H. R. Tizhoosh, "Opposition-Based Learning: A New Scheme for Machine Intelligence," *International conference on computational intelligence for modelling, control and automation and international conference on intelligent agents, web technologies and internet commerce (CIMCA-IAWTIC'06)*, vol. 1, pp. 695–701, Nov. 2005.
 - [19] Q. Kang, C. Xiong, M. Zhou, and L. Meng, "Opposition-Based Hybrid Strategy for Particle Swarm Optimization in Noisy Environments," *IEEE Access*, vol. 6, pp. 21888–21900, Mar. 2018, doi: 10.1109/ACCESS.2018.2809457.
 - [20] S. Dhargupta, M. Ghosh, S. Mirjalili, and R. Sarkar, "Selective Opposition based Grey Wolf Optimization," *Expert Systems with Applications*, vol. 151, Aug. 2020, doi: 10.1016/j.eswa.2020.113389.
 - [21] Fleury, Claude, and Vincent Braibant, "Structural optimization: a new dual method using mixed variables," *International journal for numerical methods in engineering*, vol. 23, no. 3, pp. 409–428, 1986.
 - [22] Sandgren, E., "Nonlinear integer and discrete programming in mechanical design optimization," *ASME. J. Mech. Des.*, vol. 112, no. 2, pp. 223–229, June 1990.
 - [23] Nowacki, Horst, "Optimization in pre-contract ship design," *International Conference on Computer Applications in the Automation of Shipyard Operation and Ship Design*, Aug. 1973.
 - [24] Yang, Xin-She, et al., "True global optimality of the pressure vessel design problem: a benchmark for bio-inspired optimisation algorithms," *International Journal of Bio-Inspired Computation*, vol. 5, no. 6, pp. 329–335, 2013.
 - [25] Ragsdell, K. M., and Phillips, D. T., "Optimal Design of a Class of Welded Structures Using Geometric Programming," *ASME. J. Eng. Ind.*, vol. 98, no. 3, pp. 1021–1025, Aug. 1976.